Alternative Assessment Tool (AAT-2) Report on

"Execution of Combinational Digital Modules on the Cadence Tool"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Test (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Sawan Kumar **1BF24EC136**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who helped and supported me in completing this Alternative Assessment Tool (AAT-2) report on “Execution of Combinational Digital Modules on the Cadence Tool”.

I am grateful to the Principal of BMS College of Engineering and to the Head of the Department of Electronics and Communication Engineering for providing the academic environment and the laboratory and software facilities, including the Cadence tools, needed to carry out this work.

I extend my heartfelt thanks to Prof. Ajaykumar Devarapalli, faculty of the course Advanced Digital and Logic Design (ADLD, 23ECSPE1AD), for his guidance, valuable suggestions and constant encouragement, which helped me understand how to design, compile and simulate digital circuits on the Cadence tool.

I also thank the teaching and non-teaching staff of the department, my friends for their support and discussions, and my family for their continuous motivation throughout this work.

Sawan Kumar
1BF24EC136

INDEX

INTRODUCTION **2**
Tools and Methodology 2
Modules Covered 2
Procedure on the Cadence Tool 2
MODULE 1: 4-BIT BARREL SHIFTER (COMBINATIONAL MODULE) **3**
Design Description 3
Circuit Diagram 4
Verilog HDL Code 4
Testbench 5
Simulation Waveform 5
MODULE 2: 4-BIT COMPARATOR (COMBINATIONAL MODULE) **7**
Design Description 7
Circuit Diagram 7
Verilog HDL Code 8
Testbench 8
Simulation Waveform 9
CONCLUSION **10**

INTRODUCTION

This report presents the execution of two combinational modules, a 4-bit barrel shifter and a 4-bit comparator, on the Cadence tool as part of the Alternative Assessment Tool (AAT-2) for the course Advanced Digital and Logic Design (ADLD, 23ECSPE1AD). Each module is described with its circuit diagram, Verilog HDL code, testbench and simulation waveform.

Tools and Methodology

Each module is written in Verilog HDL using the gedit editor, together with its own testbench, on a Linux workstation. The design and testbench are compiled and simulated with Cadence NCVerilog (ncverilog +access+r), and the dumped .vcd file is opened in Cadence SimVision to observe and explain the waveforms. Both modules are combinational, so no clock is used; all times are in ns (SimVision shows them in ps).

Modules Covered

No.	Module	Type	Design file	Testbench file
1	4-Bit Barrel Shifter	Combinational	barrel_shifter.v	tb_barrel_shifter.v
2	4-Bit Comparator	Combinational	comparator_4bit.v	tb_comparator_4bit.v

Procedure on the Cadence Tool

The same flow was followed for both modules.

1. Log in with the student profile and open Home, then the Cadwork folder and the Digital folder.
2. Create a folder named with the USN (1BF24EC136); inside it create a folder sim, and inside sim create the folders rtl, tb and work.
3. Open a terminal in the Digital folder. Run csh and then source .cshrc to load the Cadence environment, and check the tool with which ncverilog.
4. Go to sim/rtl, write the design with gedit filename.v, save and quit. Run ncverilog +access+r filename.v to check the design for errors.
5. Go to ../tb and write the testbench with gedit tb_filename.v. It contains $dumpfile and $dumpvars so that a .vcd file is generated.
6. Go to ../work and run ncverilog +access+r ../rtl/filename.v ../tb/tb_filename.v. Use ls to confirm that the .vcd file is created.
7. Run simvision filename.vcd. In the left pane select the testbench, then click the small waveform icon at the top right to plot the signals.

8. Observe the waveform, demonstrate the working and explain the behaviour of the module.

MODULE 1: 4-BIT BARREL SHIFTER (COMBINATIONAL MODULE)

Design Description

The barrel shifter shifts or rotates a 4-bit word by 0 to 3 positions in a single step, without a clock. The module has no memory: data_out follows data_in, shift_amt and mode through a combinational always @(*) block that selects one of four operations with a case statement on mode. Two 8-bit wires hold the data repeated twice ({data_in, data_in}), which turns the rotate operations into a simple shift of that double-width word. The 2-bit mode input selects the operation: 00 is logical shift left, 01 is logical shift right, 10 is rotate left and 11 is rotate right. In a logical shift zeros fill the vacated bits; in a rotate the bits that leave one end re-enter at the other end.

Input / output ports

Port	Direction	Width	Description
data_in	Input	4	4-bit data word to be shifted or rotated
shift_amt	Input	2	Shift / rotate amount, 0 to 3 positions
mode	Input	2	Operation select: 00 SLL, 01 SRL, 10 ROL, 11 ROR
data_out	Output	4	Shifted / rotated result

Function table (for data_in = 1011, i.e. B)

mode	Operation	amt = 0	amt = 1	amt = 2	amt = 3
00	Shift left (SLL)	1011	0110	1100	1000
01	Shift right (SRL)	1011	0101	0010	0001
10	Rotate left (ROL)	1011	0111	1110	1101
11	Rotate right (ROR)	1011	1101	1110	0111

Circuit Diagram

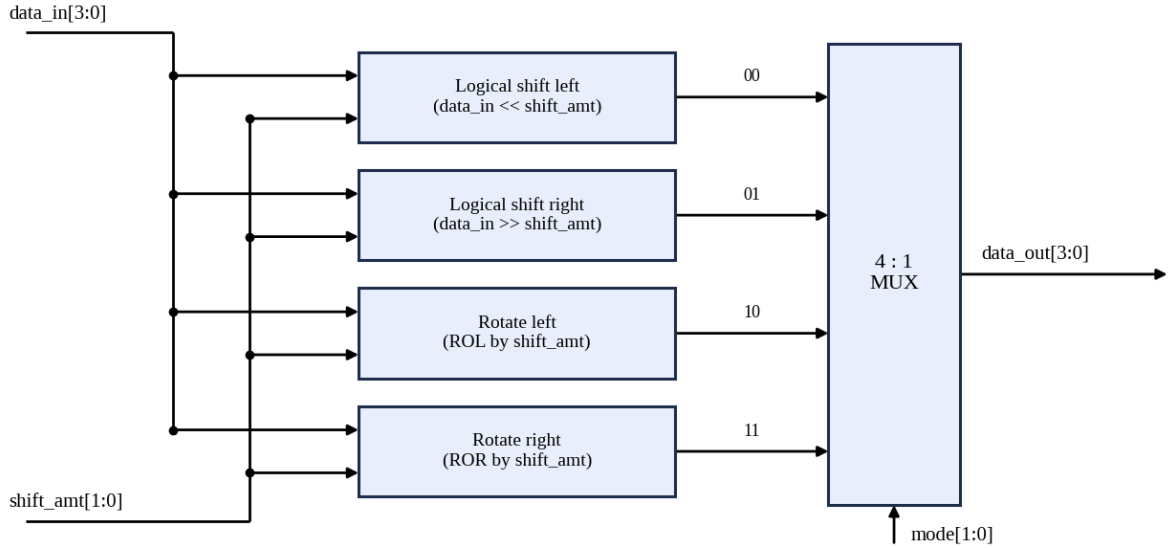


Figure 1.1: Block diagram of the 4-bit Barrel Shifter

Verilog HDL Code

The design file barrel_shifter.v was written in gedit in the rtl folder.

Open ▾ ⊞ **barrel_shifter.v**
~/cadWorkDir/Digital/1BF24EC136/sim/rtl

```
//4-bit Barrel Shifter - combinational model
`timescale 1ns/1ps

module barrel_shifter (data_in, shift_amt, mode, data_out);

input  [3:0] data_in;       // 4-bit data input
input  [1:0] shift_amt;     // shift / rotate amount (0 to 3)
input  [1:0] mode;          // 00 SLL, 01 SRL, 10 ROL, 11 ROR
output reg [3:0] data_out;  // shifted output

wire [7:0] dbl   = {data_in, data_in};  // data repeated for rotate
wire [7:0] rol_w = dbl << shift_amt;
wire [7:0] ror_w = dbl >> shift_amt;

always @(*)
begin
  case (mode)
    2'b00: data_out = data_in << shift_amt;  // logical shift left
    2'b01: data_out = data_in >> shift_amt;  // logical shift right
    2'b10: data_out = rol_w[7:4];            // rotate left
    2'b11: data_out = ror_w[3:0];            // rotate right
  endcase
end

endmodule
```

Figure 1.2: Verilog HDL code of the 4-bit Barrel Shifter in gedit

Testbench

The testbench tb_barrel_shifter.v in the tb folder applies data_in = B (1011) for 160 ns and then data_in = D (1101) for another 160 ns. For each data value it steps mode from 0 to 3 (40

ns each) and, for each mode, steps shift_amt from 0 to 3 in 10 ns steps using two nested loops. It dumps the waveform to barrel_shifter.vcd and ends the simulation at 320 ns.

Open ▾ ⊞ **tb_barrel_shifter.v**
~/cadWorkDir/Digital/1BF24EC136/sim/tb

```
//Test Bench for 4-bit barrel shifter
`timescale 1ns/1ps

module tb_barrel_shifter();

reg  [3:0] data_in;
reg  [1:0] shift_amt, mode;
wire [3:0] data_out;
integer i, d;

barrel_shifter DUT(.data_in(data_in), .shift_amt(shift_amt),
                   .mode(mode), .data_out(data_out));

initial
begin
$dumpfile("barrel_shifter.vcd");
$dumpvars(0, tb_barrel_shifter);
end

initial
begin
for (d = 0; d < 2; d = d + 1)
begin
  data_in = (d == 0) ? 4'hB : 4'hD;
  for (i = 0; i < 4; i = i + 1)
  begin
    mode = i;
    begin : blk
      integer i;
      for (i = 0; i < 4; i = i + 1)
      begin shift_amt = i; #10; end
    end
  end
end
$finish;
end

endmodule
```

Figure 1.3: Testbench of the 4-bit Barrel Shifter in gedit

Simulation Waveform

The commands used in the terminal to compile, simulate and open the waveform:

Terminal - barrel_shifter

```
% cd ~/cadWorkDir/Digital/1BF24EC136/sim/rtl
% ncverilog +access+r barrel_shifter.v
% cd ../tb
% cd ../work
% ncverilog +access+r ../rtl/barrel_shifter.v ../tb/tb_barrel_shifter.v
% ls
% simvision barrel_shifter.vcd
```

Figure 1.4: Terminal commands used for the 4-bit Barrel Shifter

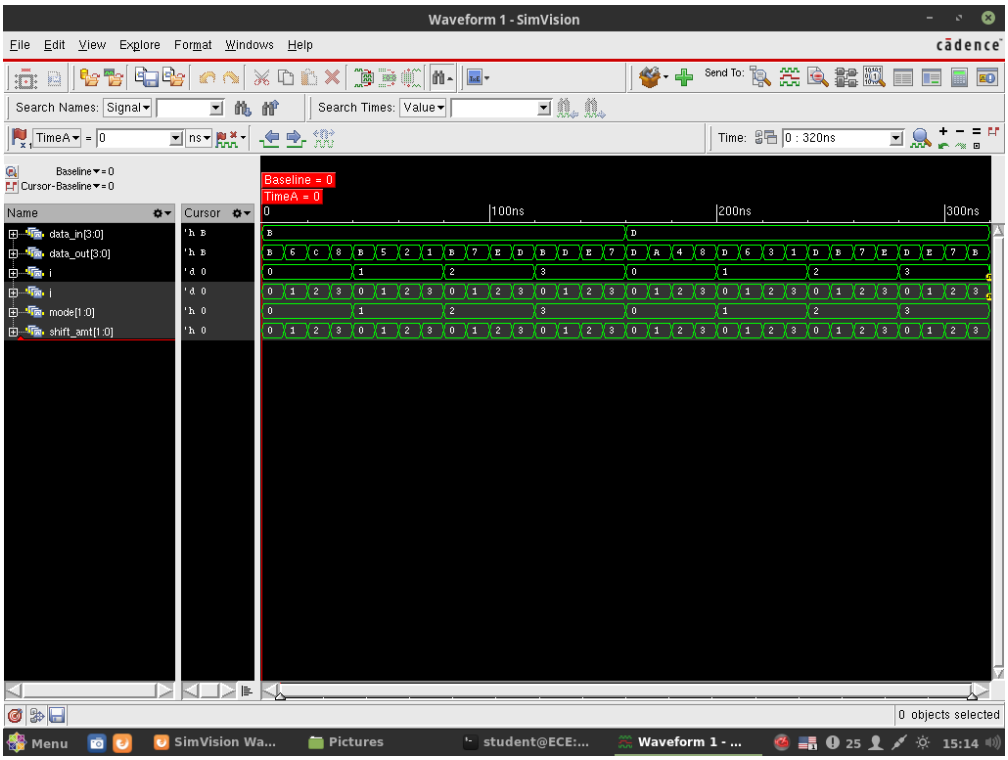


Figure 1.5: Simulation waveform of the 4-bit Barrel Shifter in SimVision

Observation: The SimVision window shows data_in[3:0], data_out[3:0], the two loop variables i of the testbench (outer loop for mode, inner loop for shift_amt), mode[1:0] and shift_amt[1:0] from 0 to 320 ns. At the cursor (0 ns) data_in = B, mode = 0 and shift_amt = 0, so data_out = B (no shift). For data_in = B (1011) the output is B, 6, C, 8 for mode 0 (shift left), B, 5, 2, 1 for mode 1 (shift right), B, 7, E, D for mode 2 (rotate left) and B, D, E, 7 for mode 3 (rotate right) as shift_amt goes from 0 to 3. At 160 ns data_in changes to D (1101) and the same sweep gives D, A, 4, 8 (shift left), D, 6, 3, 1 (shift right), D, B, 7, E (rotate left) and D, E, 7, B (rotate right). The output changes together with the inputs and no clock is involved, which confirms that the circuit is combinational.

Waveform behaviour

Time (ns)	Input event and behaviour	mode	data_out
0 - 40	data_in = B: shift left (SLL)	0	B, 6, C, 8
40 - 80	data_in = B: shift right (SRL)	1	B, 5, 2, 1
80 - 120	data_in = B: rotate left (ROL)	2	B, 7, E, D
120 - 160	data_in = B: rotate right (ROR)	3	B, D, E, 7
160 - 200	data_in changes to D: shift left (SLL)	0	D, A, 4, 8
200 - 240	data_in = D: shift right (SRL)	1	D, 6, 3, 1
240 - 280	data_in = D: rotate left (ROL)	2	D, B, 7, E
280 - 320	data_in = D: rotate right (ROR)	3	D, E, 7, B

MODULE 2: 4-BIT COMPARATOR (COMBINATIONAL MODULE)

Design Description

The 4-bit magnitude comparator compares two unsigned 4-bit numbers A and B and tells whether A is greater than, equal to or less than B. It has no clock; the outputs follow the inputs through continuous assignments that use the relational operators >, == and <. For any pair of inputs exactly one of the three outputs is high: A_gt_B is 1 when A > B, A_eq_B is 1 when A = B and A_lt_B is 1 when A < B, so the outputs are mutually exclusive.

Input / output ports

Port	Direction	Width	Description
A	Input	4	First 4-bit unsigned operand
B	Input	4	Second 4-bit unsigned operand
A_gt_B	Output	1	High when A is greater than B
A_eq_B	Output	1	High when A is equal to B
A_lt_B	Output	1	High when A is less than B

Function table

Condition	A_gt_B	A_eq_B	A_lt_B
A > B	1	0	0
A = B	0	1	0
A < B	0	0	1

Circuit Diagram

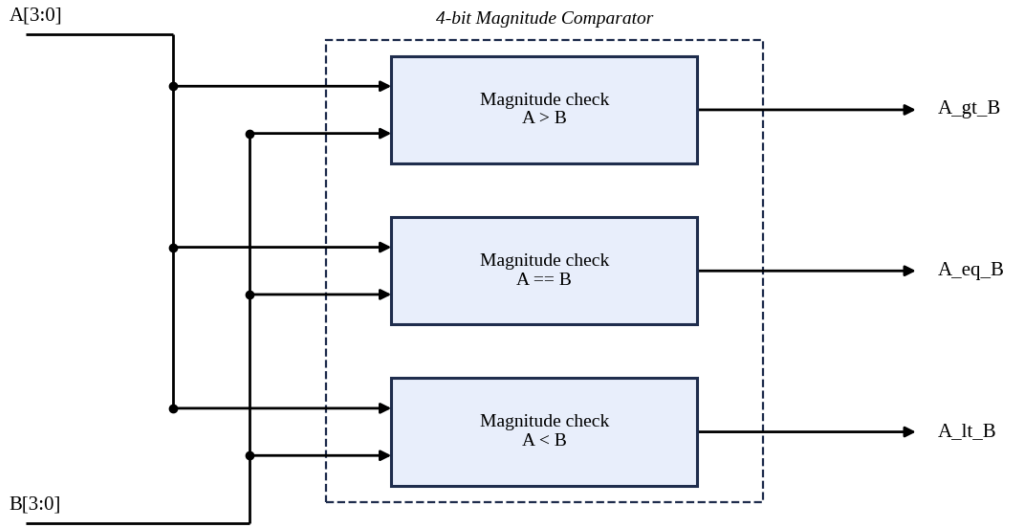


Figure 2.1: Block diagram of the 4-bit Comparator

Verilog HDL Code

The design file comparator_4bit.v was written in gedit in the rtl folder.

Open ▾ ⊞ **comparator_4bit.v**
~/cadWorkDir/Digital/1BF24EC136/sim/rtl

```
//4-bit Magnitude Comparator - combinational model
`timescale 1ns/1ps

module comparator_4bit (A, B, A_gt_B, A_eq_B, A_lt_B);

input  [3:0] A;       // first 4-bit operand
input  [3:0] B;       // second 4-bit operand
output A_gt_B;        // 1 when A > B
output A_eq_B;        // 1 when A = B
output A_lt_B;        // 1 when A < B

assign A_gt_B = (A > B);
assign A_eq_B = (A == B);
assign A_lt_B = (A < B);

endmodule
```

Figure 2.2: Verilog HDL code of the 4-bit Comparator in gedit

Testbench

The testbench tb_comparator_4bit.v applies five pairs of A and B, each held for 10 ns, in the order (5, 3), (2, 6), (9, 9), (F, 1) and (0, F), which covers the A > B, A < B and A = B cases. It dumps the waveform to comparator_4bit.vcd and ends the simulation at 50 ns.

Open ▾ ⊞ **tb_comparator_4bit.v**
~/cadWorkDir/Digital/1BF24EC136/sim/tb

```
//Test Bench for 4-bit comparator
`timescale 1ns/1ps

module tb_comparator_4bit();

reg  [3:0] A, B;
wire A_gt_B, A_eq_B, A_lt_B;

comparator_4bit DUT(.A(A), .B(B), .A_gt_B(A_gt_B),
                    .A_eq_B(A_eq_B), .A_lt_B(A_lt_B));

initial
begin
$dumpfile("comparator_4bit.vcd");
$dumpvars(0, tb_comparator_4bit);
end

initial
begin
A = 4'h5; B = 4'h3;
#10 A = 4'h2; B = 4'h6;
#10 A = 4'h9; B = 4'h9;
#10 A = 4'hF; B = 4'h1;
#10 A = 4'h0; B = 4'hF;
#10 $finish;
end

endmodule
```

Figure 2.3: Testbench of the 4-bit Comparator in gedit

Simulation Waveform

The commands used in the terminal to compile, simulate and open the waveform:

```
Terminal - comparator_4bit
% cd ~/cadWorkDir/Digital/1BF24EC136/sim/rtl
% ncverilog +access+r comparator_4bit.v
% cd ../tb
% cd ../work
% ncverilog +access+r ../rtl/comparator_4bit.v ../tb/tb_comparator_4bit.v
% ls
% simvision comparator_4bit.vcd
```

Figure 2.4: Terminal commands used for the 4-bit Comparator

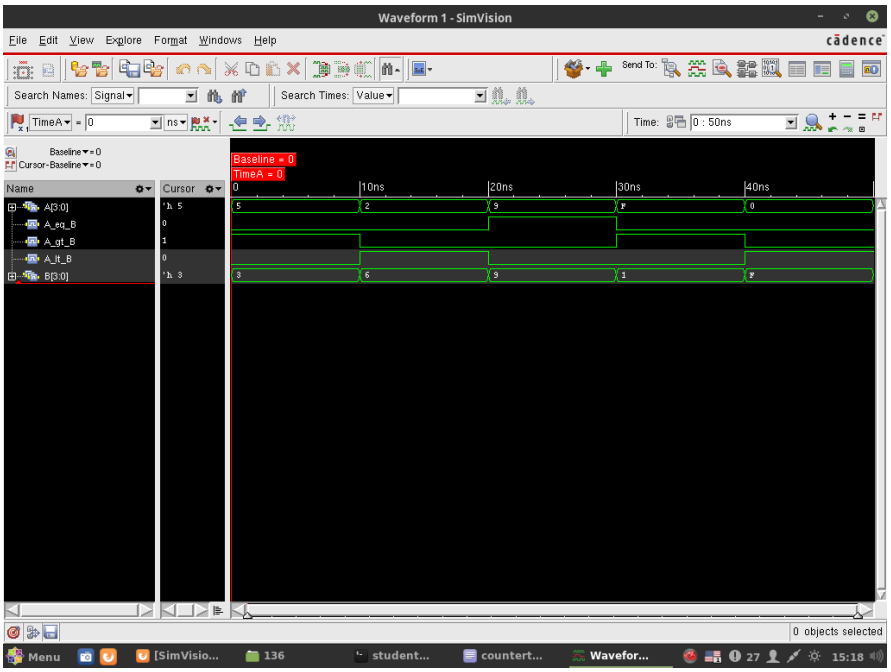


Figure 2.5: Simulation waveform of the 4-bit Comparator in SimVision

Observation: The SimVision window shows A[3:0], A_eq_B, A_gt_B, A_lt_B and B[3:0] from 0 to 50 ns. At the cursor (0 ns) A = 5 and B = 3, so A_gt_B = 1 while A_eq_B = 0 and A_lt_B = 0. A_lt_B goes high at 10 ns (A = 2, B = 6), A_eq_B at 20 ns (A = B = 9), A_gt_B again at 30 ns (A = F, B = 1) and A_lt_B at 40 ns (A = 0, B = F). Exactly one output is high in every interval and no clock is involved, so the comparator is purely combinational.

Waveform behaviour

Time (ns)	A, B (hex)	Output high	Remark
0 - 10	5, 3	A_gt_B	5 > 3: A is greater than B
10 - 20	2, 6	A_lt_B	2 < 6: A is less than B
20 - 30	9, 9	A_eq_B	Both inputs are equal
30 - 40	F, 1	A_gt_B	15 > 1: A is greater than B
40 - 50	0, F	A_lt_B	0 < 15: A is less than B

CONCLUSION

Two combinational modules, a 4-bit barrel shifter and a 4-bit comparator, were written in Verilog HDL, compiled and simulated on the Cadence tool using NCVerilog, and their waveforms were analysed in SimVision. The barrel shifter shifts or rotates the 4-bit input left or right by 0 to 3 positions as selected by mode, and the comparator raises exactly one of A_gt_B, A_eq_B and A_lt_B for every pair of inputs. In both cases the simulated behaviour matched the expected function tables.

The exercise also covered the complete Cadence flow: creating the rtl, tb and work folders, setting up the environment with csh and .cshrc, compiling with ncverilog +access+r, generating the .vcd file and viewing it in SimVision.

Module	Type	Result observed in SimVision
4-Bit Barrel Shifter	Combinational	data_out = data_in shifted or rotated by shift_amt as selected by mode; B gives B, 6, C, 8 for a left shift
4-Bit Comparator	Combinational	Exactly one of A_gt_B, A_eq_B and A_lt_B is high for each pair of A and B